

Attack/Defense Tools

SSH - VS Code - Git - Tulip
Destructive Farm - Proxy - Monke Jammer

SSH (Secure SHell): introduzione

È un protocollo che permette di stabilire una sessione remota cifrata tramite interfaccia a riga di comando con un altro host. Ovvero, ci **permette di aprire un terminale su un altro host**.

È fondamentale perché è lo strumento standard per collegarsi alla VM remota durante le competizioni di tipo Attacco/Difesa.

È possibile creare una coppia di chiavi SSH con il comando (non è necessario impostare una passphrase):

```
ssh-keygen -t ed25519 -C "your\_email@example.com"
```

SSH (Secure SHell): connessione ad host remoto

Per connettersi ad un host remoto si può usare il comando

```
ssh username@host
```

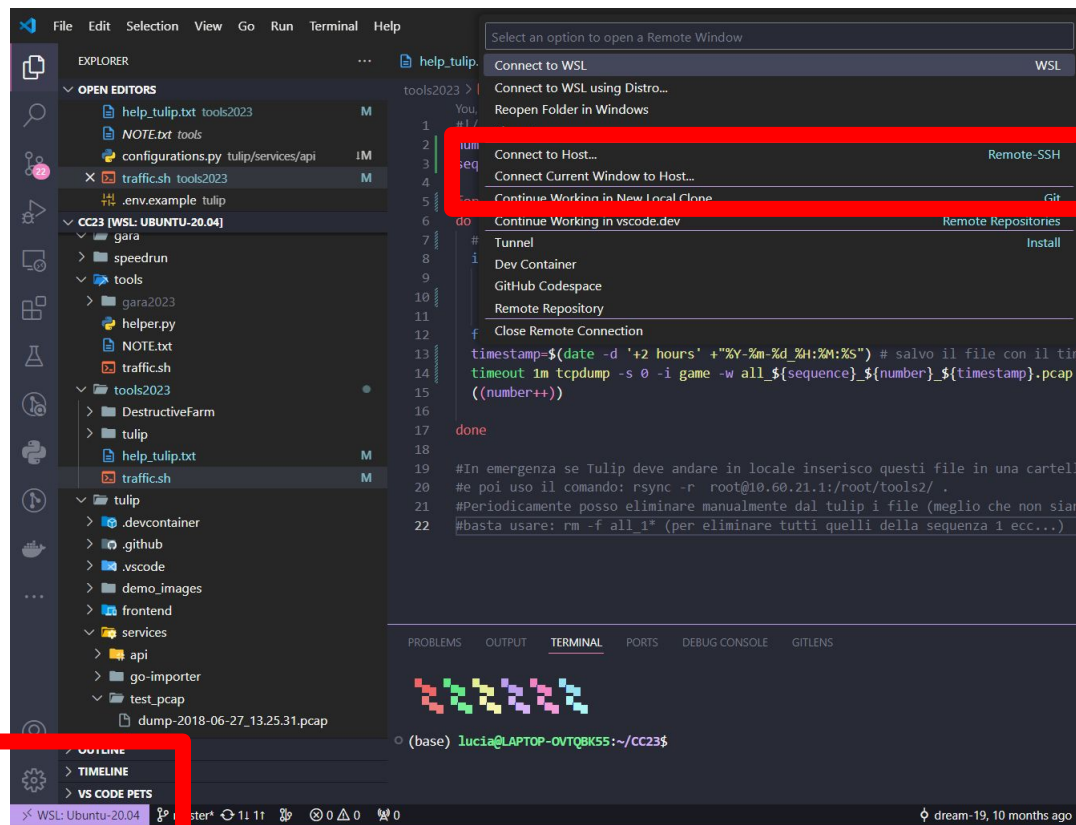
E inserire la password dell'utente “*username*”.

A livello gara generalmente è: `ssh root@ip_virtual_machine`

Oppure usando una chiave ssh con il comando (in questo caso non è necessario l'inserimento della password):

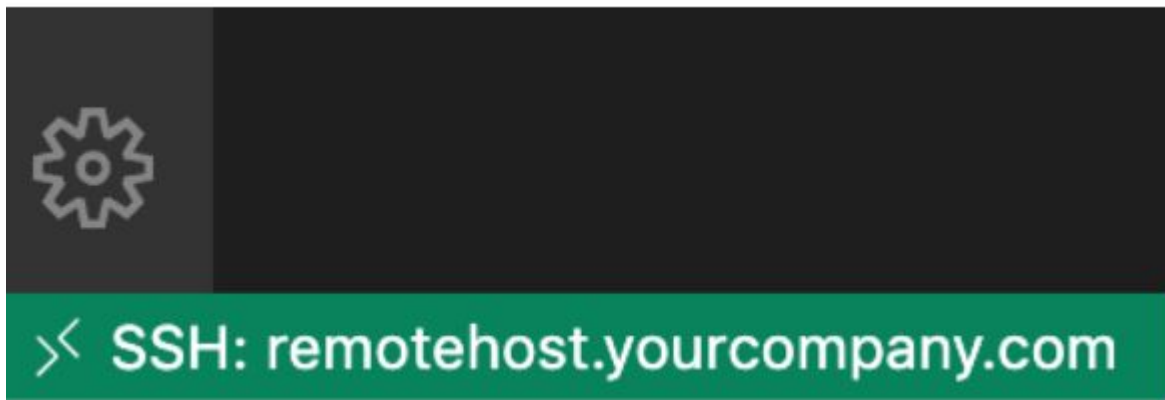
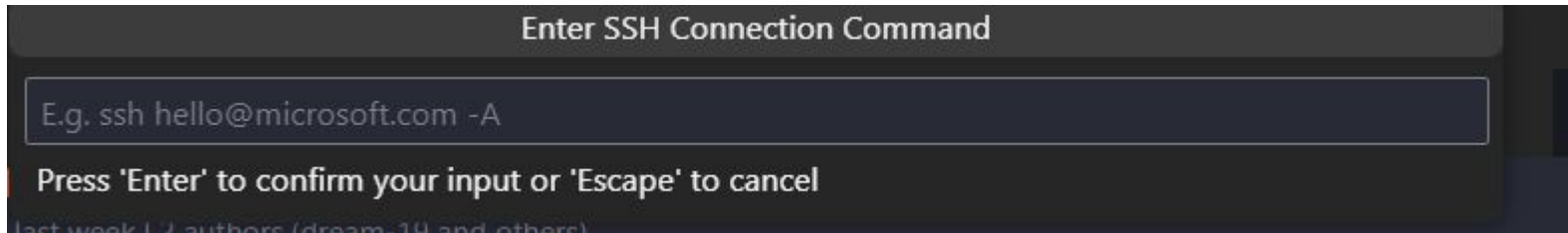
```
ssh -i ~/.ssh/key.pub username@host
```

SSH (Secure SHell): connessione ad host remoto direttamente tramite VSCODE



Utile per avere un'interfaccia per leggere/modificare i file. Senza doverlo fare da terminale

SSH (Secure SHell): connessione ad host remoto direttamente tramite VSCODE



Git: introduzione

Il codice sorgente di un'applicazione viene modificato molte volte nel corso del tempo: aggiunta funzionalità, refactoring, correzione di errori. **I sistemi di controllo di versione permettono di salvare e, se necessario, recuperare tutte le versioni significative del codice sorgente.**

git è un sistema di controllo versione sviluppo dal 2005 da Linus Torvalds (creatore del kernel Linux).



Git: definizioni

Working tree: cartella con un repository associato, indicata dalla presenza della cartella .git. Area di lavoro corrente. La cartella .git tiene traccia (localmente) di tutte le modifiche.

Commit: fotografia (snapshot) del working tree ad un certo punto del tempo.

Index: collezione di file che sono stati modificati ma non ancora assegnati ad un commit (chiamato anche **staging area**).

Branch: linea di sviluppo. Il commit più recente su un branch è chiamato estremità (tip) ed è identificata da una testa (HEAD, puntatore all'ultimo commit) che si sposta in avanti quando vengono eseguiti commit. Solitamente esistono più branch. Il branch di default è solitamente chiamato master o main.

Git: workflow

Lo sviluppo solitamente segue i seguenti passi:

1. Il lavoro viene svolto sul working tree.
2. Quando il lavoro raggiunge un punto significativo (correzione bug, integrata funzionalità, ecc) le modifiche vengono aggiunte alla staging area.
3. Quando l'index contiene materiale sufficiente, viene eseguito un commit.
4. Eventualmente, esistono più branch di sviluppo che poi vengono confrontati ed integrati.

Git: repository

Collezione di tutti i file e della loro storia (tutti i commit). Può risiedere su una macchina locale o su un server remoto (es: **GitHub**, ecc).

L'operazione di copia di un repository remoto in locale si chiama clonazione (**clone**).

L'operazione che consiste nello scaricare i commit che non esistono nel repository locale ma esistono nel repository remoto si chiama **pull**.

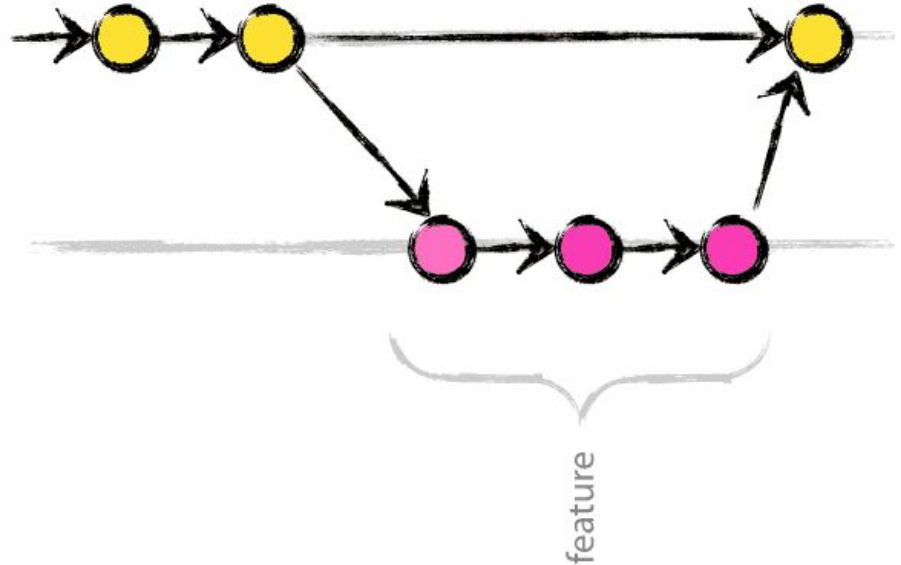
L'operazione che consiste nel caricare i commit che non esistono nel repository remoto ma esistono nel repository locale si chiama **push**.



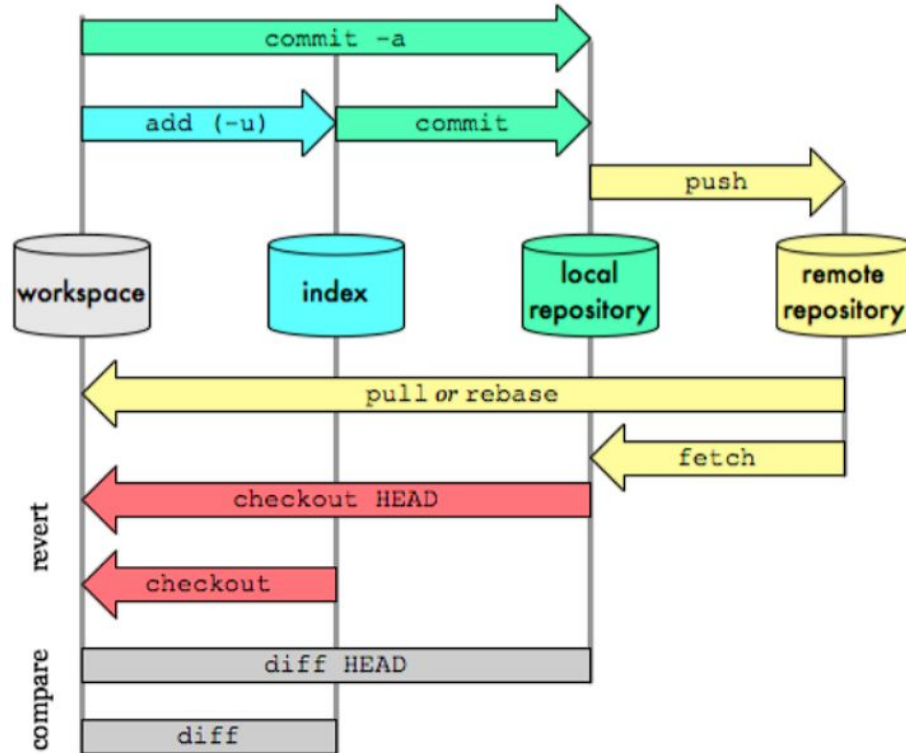
Git: branch

Tutti i commit vivono su un **branch** (ramo), esiste quindi un grafo che rappresenta la storia del codice. In questo modo possono esistere più versioni in parallelo.

Nel caso di lavoro in team ogni membro potrebbe lavorare su un branch diverso e periodicamente si effettua il **merge**, ovvero la fusione di branches differenti



Git: comandi fondamentali

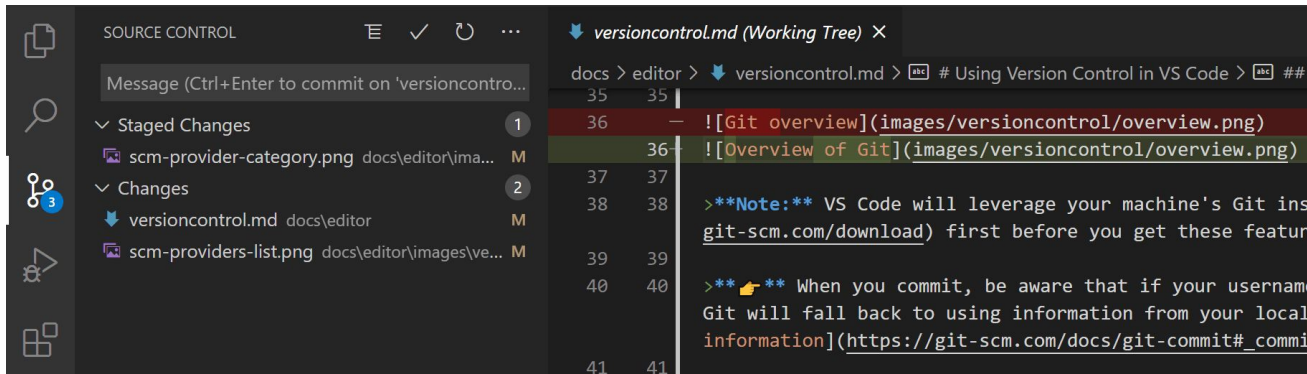


Git: estensione VS Code

Uno dei modi più semplici per usare Git (e GitHub) è quello di usare l'estensione VS Code che permette di accedere a tutte le informazioni tramite interfaccia grafica.

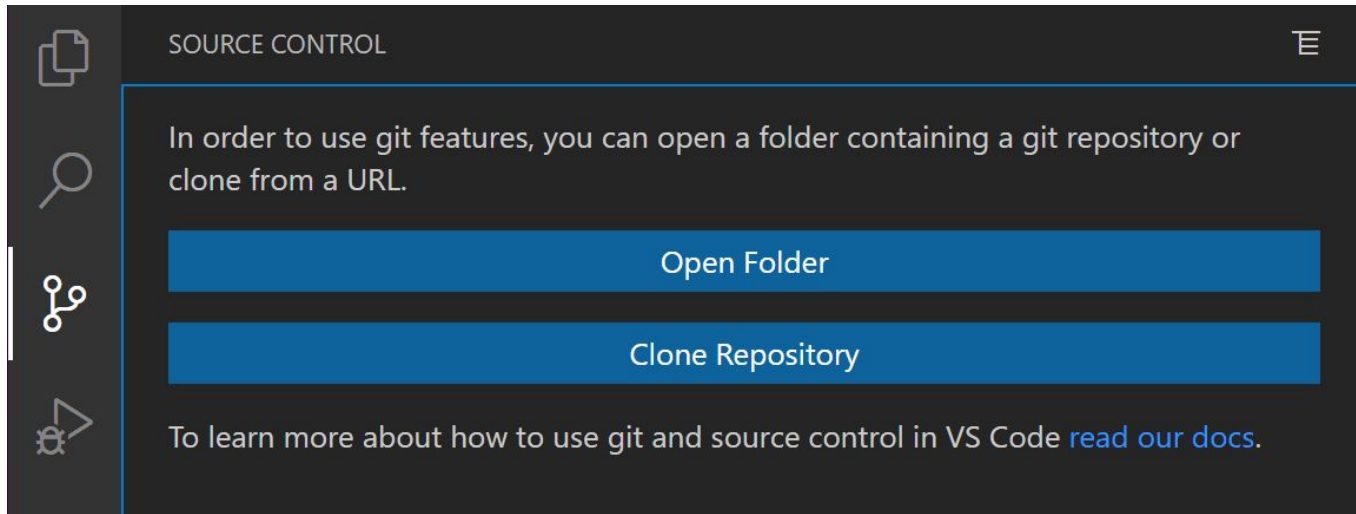
Link alla documentazione ufficiale:

<https://code.visualstudio.com/docs/sourcecontrol/overview>



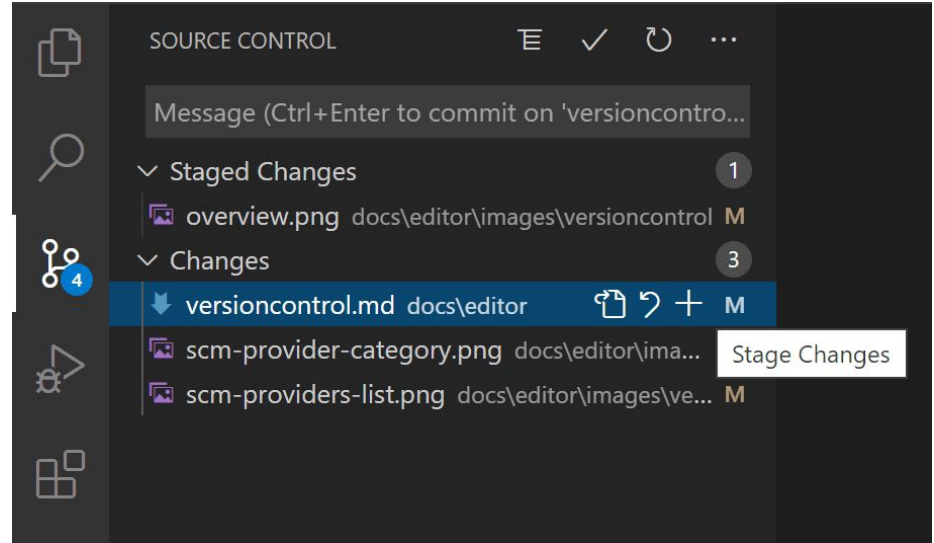
Git: estensione VS Code - clonazione repository

L'estensione permette di clonare una repository remota (es: GitHub)



Git: estensione VS Code - Commit

Per fare un commit si può premere “+” per aggiungere il file allo staging area, inserire il messaggio e premere ctrl+invio per eseguire il commit.



Git: estensione VS Code - Sincronizzazione

Se si lavora con una repository remota si può sincronizzare premendo l'apposito tasto (indicato dalle frecce) che si trova in basso a sinistra nella finestra di VS Code



Git: configurazione

Al primo utilizzo è necessario eseguire i seguenti comandi:

```
git config -- global user.name "Nome Cognome"
```

```
git config -- global user.email mioindirizzo@dominio.com
```

Spesso si parte dalla clonazione di una repository pubblica oppure se ne può creare una nuova da zero.

Per clonare repository e fare operazioni su repository private è necessario aggiungere una chiave SSH al proprio profilo seguendo la guida al link.

Git: comandi fondamentali (in vista della gara CC2024)

Aggiungere un file alla staging area

```
git add file.txt
```

Fare un commit

```
git commit -m "messaggio"
```

Cancellare le modifiche di un file

```
git restore file.c
```

Scaricare e caricare gli ultimi commit in una repository remota e applicazione delle patch ai servizi in esecuzione

```
git pull / push
```

```
docker compose up -d --rebuild
```

Git: CTF workflow

Git può essere uno strumento utile durante le competizioni CTF attacco/difesa per **tenere traccia dei cambiamenti nel codice** (le patch) e per **sincronizzare la repository locale con quella sulla VM**.

Ad inizio gara si crea una repository GitHub in cui si carica tutto il codice dei servizi.

La repository viene poi clonata sui pc di tutti i partecipanti e le ogni nuova patch viene “pushata”.

Uno dei partecipanti sarà poi incaricato di effettuare il pull delle modifiche nella VM e applicare la patch ai servizi in esecuzione. *[A meno di implementare una pipeline CI/CD]*

Tulip

<https://github.com/OpenAttackDefenseTools/tulip>

Tulip

Tulip può essere usato in due modi:

- in **locale**: su una propria macchina (possibilmente una con abbastanza ram e potenza)
- in **globale**: sulla VM della gara, in questo caso bisogna proteggerlo con una password per evitare che anche le altre squadre possano accedervi

Tulip - guida all'uso in **globale**

- 1) git clone <https://github.com/OpenAttackDefenseTools/tulip.git>
- 2) cd tulip e cp .env.example .env
- 3) in tulip > services > api > configuration.py modificare l'ip della VM e le porte dei servizi presenti (docker ps per scoprire quali sono)

```
vm_ip = "10.10.3.1"

services = [{"ip": vm_ip, "port": 9876, "name": "cc_market"},
            {"ip": vm_ip, "port": 80, "name": "maze"},
            {"ip": vm_ip, "port": 8080, "name": "scadent"},
            {"ip": vm_ip, "port": 5000, "name": "starchaser"},
            {"ip": vm_ip, "port": 1883, "name": "scadnet_bin"},
            {"ip": vm_ip, "port": -1, "name": "other"}]
```

Tulip - guida all'uso in **globale**

4) se si vuole cambiare la porta di tulip bisogna modificare ports: - 3000: 3000 del docker compose

5) lanciare Tulip: **docker-compose up -d --build** (poi si trova su localhost:3000), se c'è bisogno di ricostruire l'api finchè è in esecuzione: `docker-compose up --build -d api`

6) I tcpdump che Tulip deve leggere vanno inseriti in: `tulip>services>test_pcap`, si possono usare script per raccogliere automaticamente il traffico, come `traffic.sh`

Tulip - guida all'uso in **globale**

traffic.sh (esempio di script per raccogliere automaticamente il traffico)

- inserirlo in tulip>services>test_pcap
- chmod +x traffic.sh e poi ./traffic.sh

```
#!/bin/bash
number=1 # numero del file
sequence=1 # sequenza di appartenenza

for (( ; ; )) #ciclo infinito
do
    # Dopo 35 file (quindi ogni 35 minuti) li rimuove e inizia una nuova sequenza
    if [ "$number" -gt 35 ]; then
        number=1
        rm -f all_${sequence}_* # rimozione dei file per non intasare il disco
        ((sequence++))
    fi
    timestamp=$(date -d '+2 hours' +"%Y-%m-%d_%H:%M:%S") # salvo il file con il timestamp corrente
    timeout 1m tcpdump -s 0 -i game -w all_${sequence}_${number}_${timestamp}.pcap # creazione del pcap, ogni minuto
    ((number++))
done
```

Tulip - guida all'uso in **locale**

- 1) far partire Tulip su una macchina locale (esattamente allo stesso modo)
- 2) raccogliere il traffico sulla VM e trasferirlo in locale nella cartella test_pcap
 - a) esempio di comando usabile: **rsync -r root@IP-VM:/root/tools/pcaps/ .**

Tulip - dashboard

The screenshot shows the Tulip dashboard interface. The top navigation bar includes tabs for 'OpenAttackDefenseTools', 'Wi-Fi Login', 'DestructiveVoice/Destructive', 'Repositories', 'tools2023/DestructiveFarm', and 'Tulip'. The address bar shows the URL 'localhost:3000/flow/66164dbb4fdcd6da6cd4c8d32'. The main interface is divided into two panels.

Left Panel: Open filters

Event ID	Timestamp	Duration	Protocol	Status
scadent:8080	13:26:29.655	5ms	TCP	HTTP
cc_market:9876	13:26:28.412	411ms	TCP	FLAG-OUT
cc_market:9876	13:26:26.479	439ms	TCP	FLAG-OUT
cc_market:9876	13:26:24.621	437ms	TCP	FLAG-OUT
scadent:8080	13:26:23.057	4ms	TCP	HTTP
scadent:8080	13:26:23.046	4ms	TCP	HTTP
scadent:8080	13:26:23.037	3ms	TCP	HTTP
scadent:8080	13:26:23.026	4ms	TCP	HTTP
scadent:8080	13:26:23.015	4ms	TCP	HTTP
scadent:8080	13:26:23.004	4ms	TCP	HTTP
scadent:8080	13:26:22.993	4ms	TCP	HTTP
scadent:8080	13:26:22.981	5ms	TCP	HTTP
cc_market:9876	13:26:22.738	451ms	TCP	HTTP

Right Panel: Detailed View

Event ID: 13:26:28:479 0ms. Open in cyberchef

Buttons: Copy as pwntools, Copy as requests, Plain, Hex, Web, PythonRequest

Content:

```
[CletusAlbion] 1000 $$$.  
This is CletusAlbion shop!  
=====
```

- 1) Register a user.
- 2) Login.
- 3) Change description.
- 4) Add an item
- 5) Sell.
- 6) Buy.
- 7) Exit

```
=====
```

> Welcome to Market Board. Here you can find most precious items.
You have 1000 \$\$\$

ID	Flag	Value	Owner	Time
0)	flag_103	2147477420	by CletusAlbion	since Wed Jun 27 13:26:02 2018

```
----1DD6EVN3V8CY05R1AR5E5E0SE5Z9272=----
```

ID	Flag	Value	Owner	Time
2)	flag_99	2147475476	by KailaAlpha	since Wed Jun 27 13:18:02 2018
3)	no_flag_1	31926	by PearlinaVincent	since Wed Jun 27 13:18:39 2018
4)	no_flag_1	10877	by VaughnLeda	since Wed Jun 27 13:18:40 2018
5)	no_flag_1	849	by RichelleRoberto	since Wed Jun 27 13:18:41 2018
6)	no_flag_2	6874	by RichelleRoberto	since Wed Jun 27 13:18:41 2018
7)	flag_100	2147473392	by RetaClem	since Wed Jun 27 13:20:02 2018
8)	no_flag_1	406	by MinervaMay	since Wed Jun 27 13:20:22 2018
9)	no_flag_2	6698	by MinervaMay	since Wed Jun 27 13:20:23 2018
10)	no_flag_1	20857	by ErikMollie	since Wed Jun 27 13:20:23 2018
11)	no_flag_1	17521	by CharlotteDirk	since Wed Jun 27 13:20:24 2018
12)	flag_101	2147468216	by IssacDolly	since Wed Jun 27 13:22:02 2018
13)	flag_102	2147472143	by JoyVito	since Wed Jun 27 13:24:02 2018
14)	no_flag_1	13673	by GardnerEugenio	since Wed Jun 27 13:24:52 2018
15)	no_flag_1	346	by EmmaDonald	since Wed Jun 27 13:24:52 2018
16)	no_flag_2	20205	by EmmaDonald	since Wed Jun 27 13:24:52 2018
17)	no_flag_1	16698	by WilbertBobby	since Wed Jun 27 13:24:53 2018
25)	flag_98	2147466460	by ElmoLorraine	since Wed Jun 27 13:16:02 2018

Tulip - dashboard

The screenshot displays the Tulip dashboard interface. The top navigation bar includes tabs for 'OpenAttackDefenseTools', 'Wi-Fi Login', 'DestructiveVoice/Destructi', 'Repositories', 'tools2023/DestructiveFarr', 'Tulip', and 'Lorenzo Colombi'. The main header shows the URL 'localhost:3000/flow/66164dbb4fdc6da6cd4c8d09' and filters for 'regex', 'all', 'from', 'to', 'Last 5 ticks', and 'Graph view'. The right side of the header indicates 'First Diff ID', 'Second Flow ID', 'Diff', and 'Current: 1014673'. Below the header, there are buttons for 'Copy as pwntools' and 'Copy as requests'.

The left sidebar, titled 'Open filters', lists several events with their respective details:

- scadent:8080 13:26:29.655 5ms (TCP, HTTP)
- cc_market:9876 13:26:28.412 411ms (TCP, FLAG-OUT)
- cc_market:9876 13:26:26.479 439ms (TCP, FLAG-OUT)
- cc_market:9876 13:26:24.621 437ms (TCP, FLAG-OUT)
- scadent:8080 13:26:23.057 4ms (TCP, HTTP)
- scadent:8080 13:26:23.046 4ms (TCP, HTTP)
- scadent:8080 13:26:23.037 3ms (TCP, HTTP)
- scadent:8080 13:26:23.026 4ms (TCP, HTTP)
- scadent:8080 13:26:23.015 4ms (TCP, HTTP)
- scadent:8080 13:26:23.004 4ms (TCP, HTTP) - Selected
- scadent:8080 13:26:22.993 4ms (TCP, HTTP)
- scadent:8080 13:26:22.981 5ms (TCP, HTTP)
- cc_market:9876 13:26:22.738 451ms (TCP, FLAG-OUT)

The main content area shows the details of the selected event (scadent:8080 13:26:23.004 4ms). It includes a 'Meta' section with the source path '/traffic/dump-2018-06-27_13.25.31.pcap' and tags '[tcp, http]'. The source-target range is '10.10.3.126:44238 - 10.10.3.1:8080'. The event is marked as 'FLAG-OUT' and has a status of '0ms'. A button 'Open in cyberchef' is available. The event details are displayed in a 'Plain' view, showing the following HTTP request and response:

```
GET /list.php?pump_id=3 HTTP/1.1
Host: 10.10.3.1:8080
User-Agent: python-requests/2.18.4
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive

HTTP/1.1 200 OK
Date: Wed, 27 Jun 2018 11:26:23 GMT
Server: nginx/1.14.0 (Ubuntu)
Content-Type: text/html; charset=UTF-8
Content-Length: 0
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
```

The bottom status bar shows system information: CPU usage (15%, 04%), memory usage (33.2°C), network speed (0/0 MB/s), battery level (95%), and system time (10 apr 10:40:31).

Destructive Farm

Destructive Farm è un exploit manager per competizioni di CTF (Capture The Flag) di attack-defense. Permette di automatizzare il processo di cattura e sottomissione delle flag. È uno **strumento fondamentale durante le competizioni di tipo attacco/difesa.**

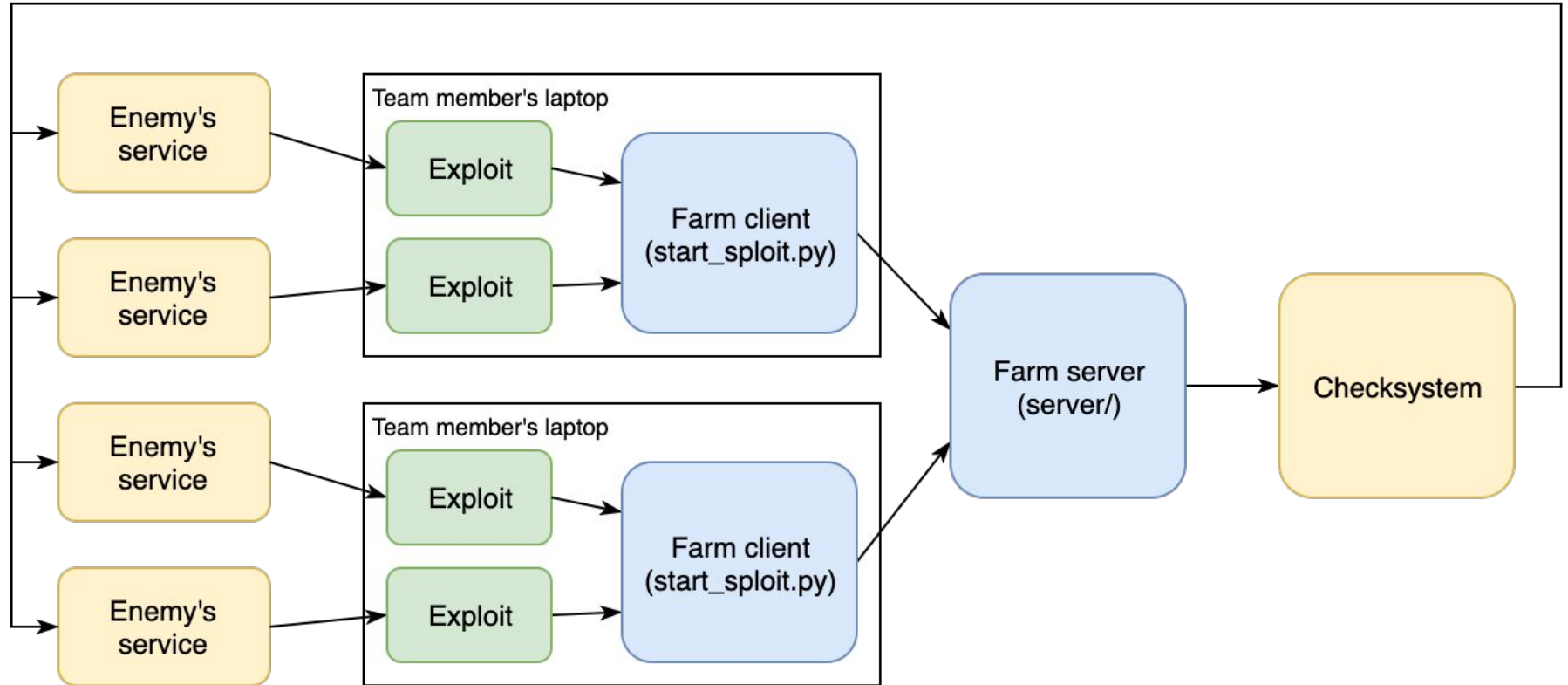
Link alla repository ufficiale: *<https://github.com/DestructiveVoice/DestructiveFarm>*

Destructive Farm: architettura

Destructive Farm è un exploit manager per competizioni di CTF (Capture The Flag) di attack-defense. È composto da:

- Exploit: script che cattura flag agli avversari. Deve prendere **in input l'indirizzo IP del bersaglio come primo parametro e stampare la flag catturata nello standard output.**
- Farm client: è uno script chiamato *start_sploit.py* che viene eseguito sui pc dei partecipanti e periodicamente esegue gli exploit
- Farm server: raccoglie le flag dai client e le sottomette al server di gara. Inoltre monitora l'andamento e mostra le statistiche in una interfaccia web. **Deve essere opportunamente configurato** e viene eseguito sul pc di un partecipante o sulla VM di gara.

Destructive Farm: architettura



Destructive Farm: farm server

Destructive Farm - Server

farm.kolambda.com:5000

Destructive Farm

Show Flags

Spoit

All

Team

All

Flag

substring, case-insensitive

Since

UTC+05

yyyy-mm-dd hh:mm

Until

UTC+05

yyyy-mm-dd hh:mm

Status

All

Checksystem response

Show

Add Flags Manually

Text with flags

flag format: [A-Z0-9]{31}=

Send

Found 8432 flags

1

2

3

4

>

Spoit	Team	Flag	Time	Status	Checksystem Response
spl_nodbrary_simple.py	c00kies@venice	8MN5ZIM7EYIWRQYQVX1NCQRL0ZHK5ZV=	2018-04-15 21:00:21	REJECTED	Denied: invalid flag
spl_lifesim_fast_50.py	SiBears	74O107CNV8JVUV5J3DAT86C2213F581=	2018-04-15 21:00:20	ACCEPTED	Accepted. 17.2257363706221 flag points
spl_lifesim_fast_50.py	SwissMadeSecurity	2NYZS1JRY3R6H8RU7GC8U5A33885C25=	2018-04-15 21:00:20	ACCEPTED	Accepted. 1.82074009957198 flag points
spl_lifesim_fast_50.py	ENOFLAG	PUUNF9JAE54T71FXO354H35CFE55614=	2018-04-15	ACCEPTED	Accepted. 6.03592550857279

Destructive Farm: guida client

La configurazione del client si esegue direttamente da linea di comando con dei parametri e non ha un file di configurazione dedicato:

```
start_sploit.py exploit_file -u SERVER_IP -a ATTACK_NAME_OR_SERVICE_NAME --token TOKEN (Il token solo se si ha attivato il login delle api, disattivato di default)
```

Destructive Farm: guida server

La configurazione del server è contenuta nel file *config.py* in cui bisogna indicare il protocollo usato per la sottomissione e il formato delle flag.

Inoltre, è necessario configurare i team che partecipano del formato “*team_id* : *team_ip*” e la password per accedere alla dashboard web.

Infine, Nel file *config.py* bisogna configurare le tempistiche della gara, in particolare *SUBMIT_FLAG_LIMIT*, *SUBMIT_PERIOD* e *FLAG_LIFETIME* (secondi)

Proxy

Nel contesto della navigazione è un server che funge da **intermediario tra l'utente e un servizio**. Quando si utilizza una proxy, il traffico viene indirizzato attraverso questo server intermediario.

La proxy ha scopo di **filtrare, scartando o modificando i pacchetti in ingresso**.

Per questo motivo può essere usata durante la competizione per **bloccare alcuni tipi di attacchi in attesa della patch**.

In questo modo la difesa diventa molto più veloce e non è necessario capire come gli avversari stanno eseguendo l'attacco ma basta identificare un passaggio che porta alla cattura della flag. Ad esempio se si nota analizzando il traffico che una certa stringa viene usata nell'exploit si può usare la proxy per scartare le richieste che la contengono.

Proxy - caveat

Filtri troppo restrittivi potrebbero bloccare la richieste legittime e quindi il server di gara potrebbe credere che il servizio non sia attivo con una **conseguente perdita di punti SLA.**

Quindi la proxy è uno strumento non obbligatorio che può essere utile ma va usato con molta cautela.

Infine, alcuni servizi presenti durante la competizione sono espressamente pensati per impedire l'uso di una proxy.

Proxy - ctf_proxy

ctf_proxy è sviluppata espressamente per le competizioni CTF ed è facilmente configurabile attraverso script Python. È un progetto open source basato su container Docker.

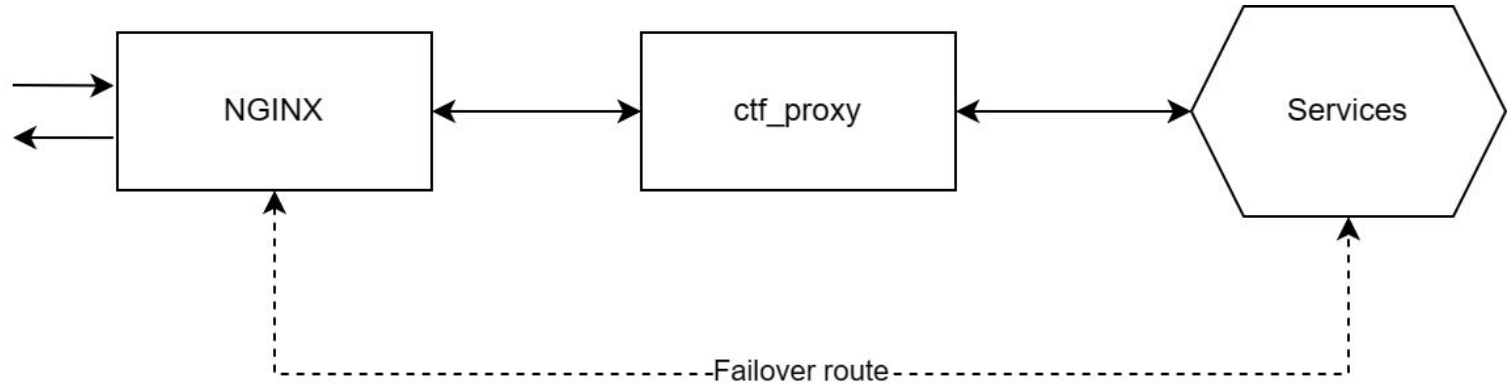
Link alla repository: https://github.com/ByteLeMani/ctf_proxy?tab=readme-ov-file

Permette di definire regole sia HTTP che TCP. Inoltre, i filtri vengono modificati automaticamente ad ogni modifica.

Essendo pensato specificatamente per la CyberChallenge può essere installata con un singolo comando:

```
pip install ruamel.yaml; curl -s  
https://raw.githubusercontent.com/ByteLeMani/ctf_proxy/main/setup_proxy.py > setup_proxy.py;  
python3 setup_proxy.py
```

Proxy - ctf_proxy - architettura



Comandi sempre utili per la gara

- Link ai **flagID**: <http://10.10.0.1:8081/flagIds>
- Per vedere i servizi presenti e su quali porte si trovano

docker ps

- copiare tutti i file di una cartella da remoto a locale (comando **scp** o **rsync**)

scp -r root@10.60.21.1:/root .

copia tutti i file presenti nella vm nella cartella root e li mette nella cartella corrente

- copiare tutti i file da locale a remoto

scp -r /percorso/file_locale root@10.60.21.1:/percorso/destinazione_remota

- ricaricare i servizi dopo averli modificati

docker compose up -d --build